

RansKnow: A Weakly-Labeled Corpus of Ransomware-Relevant YouTube Transcripts with an Audited Label-Quality Baseline

Henry Kabuye
[affiliation]

Abstract

[TODO] Write after Phase 1 baselines exist. Should state: (1) what RansKnow is (1,128 video records, 1,034 transcripts, 92 cybersecurity YouTube channels, MITRE ATT&CK-aligned regex features); (2) the paper’s actual contribution, which is arguably the label-quality audit and gold evaluation subset as much as the corpus itself – Section 4 found that a naive keyword-matched family label collided with ordinary English words (`play`, `hive`, `cactus`, ...) badly enough to inflate the single most-detected ransomware family by $\sim 3.5\times$; (3) headline baseline/calibration/OOD numbers once Phase 1+ is complete.

1 Introduction

[TODO] The gap: no open, transcript-level ransomware knowledge corpus with MITRE ATT&CK-aligned annotations. Motivate why YouTube security content specifically (volume, diversity of source – vendor threat-intel channels, DFIR conferences, independent researchers – versus e.g. only vendor blog posts). State the paper’s contributions as a list: (1) the corpus, (2) the label-quality audit methodology and its concrete findings, (3) the experiment framework and baseline results, (4) [uncertainty/conformal/OOD once those phases exist].

2 Related Work

[TODO] Three threads: (a) existing ransomware/threat-intel datasets (structured IOC feeds, MITRE ATT&CK mapping efforts, prior ransomware incident corpora – note what’s specific to RansKnow: raw natural-language transcripts rather than structured feeds); (b) weak/distant supervision in security NLP (keyword/regex-based labeling, its known failure modes); (c) conformal prediction and OOD detection in NLP, as background for Sections 7 and 8 once written.

3 Dataset Construction

RansKnow [Kabuye, 2026] consists of 1,128 video records sourced from 92 cybersecurity YouTube channels, of which 1,034 have an extracted transcript with real content; the remaining 94¹ could

1

[TODO] Verify this figure against the live dataset before submission – $1128 - 1034 = 94$, but confirm this is failed transcript extraction and not something else, e.g. videos excluded post-fetch.

not be transcribed by any method in the fetch cascade.

3.1 Channel selection

Channels were curated in two batches. Batch 1 (50 channels, C01–C50) covers established DFIR vendors, security conferences, and independent researchers. Batch 2 (51 candidate channels, C51–C101) expanded coverage into threat-intelligence vendors, additional conferences, and OT/ICS-security and government channels; 42 of the 51 candidates yielded usable transcripts, 1 was a confirmed duplicate of an existing Batch 1 channel, 4 had no discoverable dedicated YouTube channel, and 4 resolved to real channels with zero ransomware-keyword-matching content in their upload history.

[TODO] Cite the specific channel registry files (`rubrics/Dataset_Channel_Registry*.xlsx`) and consider a full channel table as a supplementary table rather than inline.

3.2 Video selection

Within each channel, videos were selected by scanning recent uploads for a fixed keyword list (`ransomware`, `extortion`, `lockbit`, `incident response`, ...; full list in Appendix A) matched against title and description, capped at 20 videos per channel and a minimum duration of 5 minutes.

[TODO] Report the actual keyword list and discuss selection bias explicitly: this is a convenience sample conditioned on channel operators’ own titling and description-writing behavior, not a random sample of ransomware discourse.

3.3 Transcript acquisition

Transcripts were acquired via a three-stage cascade: (1) the YouTube Data API’s official captions where available; (2) `yt-dlp`’s auto-generated captions; (3) local Whisper transcription as a fallback. **This cascade matters more than a footnote:** only 74 of 1,034 transcripts (7.2%) came from official captions. 959 (92.7%) are ASR output – 554 from `whisper_base` (the smallest, least accurate Whisper checkpoint) and 405 from a local Whisper fallback, 1 from `yt-dlp` captions.

[TODO] This is Finding B from the label-quality audit (Section 4). Decide whether to present it once, here, or duplicate the framing in both sections – probably present the raw fact here and the downstream implication (jargon mistranscription risk) in Section 4.

4 Label-Quality Audit

The Knowledge Agent pipeline (`Scripts/knowledge_agent.py`) extracts structured features from every transcript via regex matching: ransomware family mentions (against a maintained alias list), MITRE ATT&CK tactic mention counts across 10 tactics, offensive tool mentions across 8 tools, and platform signals (Windows/Linux/ESXi). Three concrete problems were found and are described here rather than left as an implicit caveat, because a downstream classifier trained on these labels inherits whatever bias they contain.

4.1 Finding A: generic-word alias collisions

Nine of 41 family aliases collide with common English or business words (`play`, `hive`, `cactus`, `agenda`, `cl0p`, `inc`, `maze`, `medusa`, `royal`) – identified programmatically via dictionary lookup against `/usr/share/dict/words`, then confirmed by manual transcript inspection (e.g. `hive` matching “registry hive,” a routine DFIR term, rather than Hive ransomware). A windowed-context disambiguation fix (requiring a ransomware-context term within ~ 15 tokens of the match) was applied and reduces false-positive volume substantially without eliminating it – residual false positives still occur when context vocabulary is itself dense (see `Scripts/audit_family_aliases.py` for the audit methodology and Table 1 for before/after counts).

Table 1: Before/after family mention counts for the 9 flagged aliases, windowed-context disambiguation applied. `Play` alone accounted for 386 of 494 (78%) of all family-tagged videos before the fix.

Family	Before	After	Dropped
Play	386	111	275
Qilin	88	21	67
Hive	23	10	13
Maze	11	5	6
Royal	28	19	9
INC	10	7	3
Cactus	9	6	3
Cl0p	11	11	0
Medusa	7	7	0

4.2 Finding B: ASR-dominated transcript quality

As noted in Section 3.3, 92.7% of transcripts are ASR output rather than official captions.

[TODO] Run the Phase 0.4 spot-check (manual comparison of ASR output against source audio for a fixed jargon term list – `LockBit`, `Mimikatz`, `PsExec`, `Cobalt Strike`, `rclone`, `BloodHound` – across the whisper-transcribed subset) and report a measured word-error-rate-on-jargon figure here. Aggregate detection-rate comparisons across `Transcript Provider` did not show an obvious gap, which is not the same as no effect – individual mistranscriptions of rare jargon can matter without moving an aggregate rate.

4.3 Gold evaluation subset

A stratified sample of 180 videos (channel type $\times$ year bucket $\times$ family-count bucket) was drawn for independent human annotation, held out from all model training.

[TODO] Report once annotation is complete: sample size actually annotated, Cohen’s κ per field (ransomware-relevant Y/N, family, dominant tactic, platform), adjudication process, and confusion between Knowledge Agent labels and adjudicated gold labels. This is the section that turns every later result from “matches a regex” into “matches human judgment.”

5 Task Definitions and Baselines

Five prediction tasks are defined over the corpus: ransomware family (multi-label, 18 classes after excluding 3 singleton-occurrence classes, see Table 2), dominant MITRE tactic (multi-class, 11 classes), platform signal (multi-class, 4 classes), offensive tool mentions (multi-label, 7 classes after excluding MegaNZ, which has zero occurrences in the corpus), and a ransomware-relevance binary screen (derived, not a stored column: `Family_Count > 0 OR Tactic_Impact > 0`).

Table 2: Task taxonomy.

Task	Type	Classes	Label source
Family	multi-label	18	alias regex match (audited, §4.1)
Dominant tactic	multi-class (+none)	11	max of 10 tactic-pattern counts
Platform signal	multi-class (+none)	4	max of 3 platform-pattern counts
Tool mentions	multi-label	7	per-tool regex match
Ransomware-relevant	binary	2	derived

5.1 Model ladder and validation protocol

Each task is evaluated over three feature representations (structured Knowledge Agent columns, TF-IDF, embeddings²) and five models (the rule-based Knowledge Agent itself as baseline 0, Logistic Regression, Linear SVM, Random Forest, LightGBM), under three cross-validation protocols: stratified random k -fold, GroupKFold by `Channel_ID` (to detect channel-style leakage – a model can learn “this is CrowdStrike’s narration style” instead of the content), and a temporal split (train pre-2024, test 2024+, since 79% of the corpus is from 2024 onward).

[TODO] Results table goes here. A first structured-feature pass exists (`outputs/experiments/phase1_struct`) the rule-based baseline scores a tautological macro-F1 of 1.000 on both `family` and `dominant_tactic` (expected – checked against its own labels, not independent evidence of anything). Trained models reach 0.81–0.83 macro-F1 on `dominant_tactic` from structured features alone (GBM), but top out under 0.08 on `family` from the same features – family identity is not well captured by tactic/tool count patterns, motivating the TF-IDF/embedding representations for that task specifically. **None of these numbers are gold-set-verified yet** (§4.3) and should not be reported as the paper’s headline results until they are; they’re included here as a framework-validation artifact, not a finding.

6 Uncertainty Quantification

[TODO] Not implemented yet (Phase 2). Plan: reliability diagrams + Expected Calibration Error per task/model, scored against the gold set (not the weak labels a model that’s perfectly calibrated against its own training signal has just learned to be confidently correlated with a regex). Random

²

[TODO] Embedding representation is currently blocked in the reference implementation: `sentence_transformers` fails to import due to a torch/transformers version mismatch. Resolve before this table can include embedding-representation numbers.

Forest tree-vote entropy as a free uncertainty proxy; MC-Dropout / small deep ensemble for the eventual transformer model. Headline metric: confidently-wrong rate on the gold set.

7 Conformal Prediction

[TODO] Not implemented yet (Phase 3). Plan: split conformal on `dominant_tactic/platform` at 90% target coverage; per-label conformal calibration (or conformal risk control) for `family/tool`. Conditional-coverage stress test on the gold subset, post-2025 videos, and conference-channel videos – coverage collapse on any slice is itself the result, and feeds directly into the OOD setups below via the nonconformity score.

8 Out-of-Distribution Detection

[TODO] Not implemented yet (Phase 4). Four setups already grounded in real splits: temporal emergence (train pre-2024, test on Qilin/DragonForce/Black Basta, which only appear post-2023 in this corpus), channel-type shift (Vendor+DFIR vs. Conference+Independent), leave-rare-family-out, and an external non-security corpus as a sanity check. Methods to compare: max-softmax-probability, Mahalanobis distance on pooled embeddings, energy score, and the Section 7 nonconformity score used directly as an OOD flag.

9 Interpretability and Perturbation Robustness

[TODO] Not implemented yet (Phase 5). Confound checks (length normalization, channel-identity masking, ASR-provider-stratified error analysis) plus two novel-to-this-paper diagnostics: (1) counterfactual explanations – greedy word-removal search asking whether a trained classifier’s Play predictions bottom out on removing the word “play” itself with no other family-specific vocabulary in the minimal edit set, which would show Finding A’s noise propagated past training rather than being learned away; (2) an untargeted perturbation-robustness sweep (simulated ASR noise on the 74 official-caption videos, synonym substitution) as the dataset-appropriate proxy for adversarial robustness – there is no real adversary submitting transcripts to evade this classifier in production, so a full gradient-based adversarial-attack framework is disproportionate; the actual risk surface is ASR noise and channel-style variation. Combined with Section 8’s temporal-emergence and leave-rare-family-out setups, these three pieces together are the paper’s answer to “how does this handle evolving, new, and adversarial patterns,” not three separate open questions.

10 Limitations and Ethics

[TODO] Expand into full prose; bullets below are the content, not the final form.

- **Sampling bias.** The corpus is conditioned on which channels operators chose to run, which videos they chose to title and describe in keyword-matchable ways, and YouTube’s own recommendation and search surfacing – not a random or exhaustive sample of ransomware-relevant discourse.

- **ASR noise.** 92.7% of transcripts are machine-transcribed, mostly by `whisper.base`. Section 4’s Finding B and the pending jargon spot-check (Section 4.3) bound how much this matters.
- **Weak-label ceiling.** Every quantitative result prior to gold-set verification is bounded by the Knowledge Agent’s own precision/recall, which Finding A shows is not perfect even after the disambiguation fix.
- **Dual use.** This is a corpus of ransomware tactics, tools, and case studies extracted from public educational content.

[TODO] Add a considered dual-use statement – the underlying source material is already public (YouTube videos aimed at defenders), but structuring it may lower the effort to extract offense-relevant patterns; discuss how this compares to existing public MITRE ATT&CK mappings and threat-intel reporting, which already aggregate similar information.

11 Conclusion

[TODO] Write last. Should state the gold-eval subset and audit code as a second contribution alongside the transcripts themselves, not just a validation step – other weak-label security-NLP datasets built the same way (keyword/regex extraction over scraped text) can reuse the audit methodology, not just the corpus.

References

Henry Kabuye. RansKnow: A ransomware knowledge dataset. <https://www.kaggle.com/datasets/henrykabuye/ransknow-v1>, 2026. Accessed 2026-08-11.

A Keyword List

[TODO] Insert the full KEYWORDS list from `Scripts/fetch_transcripts.py` verbatim.